



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Tema de la guía práctica proporcionada por el docente
Unidad de Organización Curricular:	Unidad Básica
Nivel y Paralelo:	Primero - B
Alumnos participantes:	Atiencia Cherres Josué Alexander Barrionuevo Montesdeoca Job Gabriel Rocha Rocha Carolina Estefania Segovia Garcia Joseph Andre
Asignatura:	Algoritmos y lógica de programación
Docente:	Ing. José Caizabuano

Link de Git Hub:

<https://github.com/JosueAtiencia/T2.6-Operaciones-Avanzadas-e-Integridad-de-Datos>

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar programas utilizando algoritmos y estructuras básicas de programación para resolver problemas mediante búsqueda lineal y validación de datos, aplicando pseudocódigo, diagramas de flujo y programación en C++ o Java de forma ordenada y funcional.

Específicos:

- Implementar un algoritmo de búsqueda lineal que permita encontrar productos dentro de un arreglo y mostrar su información correspondiente.
- Aplicar validaciones de entrada de datos para prevenir errores como el desbordamiento de caracteres en cadenas de texto
- Desarrollar soluciones utilizando pseudocódigo, diagramas de flujo y programación estructurada para fortalecer la lógica de programación.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales:1

No presenciales: 2

2.4 Instrucciones

Detalladas en la guía proporcionada por el docente.

2.5 Listado de equipos, materiales y recursos



Listado de equipos y materiales generales empleados en la guía práctica:

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☐ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☐ Inteligencia Artificial

Otros (Especifique): Github Desktop, Github Web, Editor de código Visual Studio Code.

2.6 Actividades por desarrollar

Detalladas en la guía proporcionada por el docente.

2.7 Resultados obtenidos

Se logró implementar correctamente el algoritmo de búsqueda lineal para localizar productos dentro de un arreglo.

Se validó adecuadamente la cantidad de caracteres ingresados por el usuario para evitar exceder el límite permitido.

Los programas desarrollados funcionaron correctamente en PSeInt y en el lenguaje de programación seleccionado.

Se fortaleció el uso de estructuras condicionales, ciclos, arreglos y validaciones de datos.

Se comprendió la importancia de organizar algoritmos mediante pseudocódigo y diagramas de flujo antes de programar.

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☒ Adaptabilidad
- ☐ Responsabilidad

2.9 Conclusiones

1. La búsqueda lineal permite encontrar elementos de manera sencilla dentro de un arreglo recorriendo posición por posición.
2. Las validaciones de datos son importantes para evitar errores en los programas y mejorar la seguridad de la información ingresada.



3. El uso de pseudocódigo y diagramas de flujo facilita la comprensión y desarrollo de soluciones algorítmicas antes de implementarlas en un lenguaje de programación

2.10 Recomendaciones

1. Validar siempre los datos ingresados por el usuario para evitar errores durante la ejecución del programa.
2. Utilizar nombres claros y organizados para variables y algoritmos con el fin de mejorar la comprensión del código.
3. Realizar pruebas constantes del programa para comprobar que todas las funcionalidades trabajen correctamente.

2.11 Anexos

Ejercicio 1: Búsqueda Lineal de Productos

Desarrollar un programa en C++ o Java que permita buscar un producto dentro de un arreglo utilizando el algoritmo de búsqueda lineal.

Indicaciones:

Crear un arreglo con 5 nombres de productos.
Crear otro arreglo paralelo con sus precios.
Solicitar al usuario el nombre del producto a buscar.
Recorrer el arreglo usando búsqueda lineal.

Mostrar:

Nombre del producto.
Precio correspondiente.
Utilizar break cuando se encuentre el producto.
Mostrar un mensaje si el producto no existe.

1. ANÁLISIS DEL PROBLEMA

El programa debe permitir buscar un producto dentro de un arreglo utilizando el método de **búsqueda lineal**.

Entrada

- El usuario ingresa el nombre del producto que desea buscar.

Proceso

1. Crear un arreglo con 5 nombres de productos.
2. Crear otro arreglo paralelo con los precios de cada producto.
3. Recorrer el arreglo desde la primera posición hasta la última.



4. Comparar el nombre ingresado con cada producto.
5. Si el producto se encuentra:
 - Mostrar el nombre del producto.
 - Mostrar su precio.
 - Utilizar break para detener la búsqueda.
6. Si termina el recorrido y no se encuentra el producto:
 - Mostrar un mensaje indicando que no existe.

Salida

- Nombre y precio del producto encontrado.
- O mensaje: “Producto no encontrado”.

2. DECLARACIÓN DE VARIABLES

Tipo de dato	Nombre de la variable	Descripción
string	productos[5]	Arreglo que almacena los nombres de los productos
float	precios[5]	Arreglo paralelo que almacena los precios de los productos
string	buscar	Guarda el nombre del producto que el usuario desea buscar
bool	encontrado	Indica si el producto fue encontrado o no
int	i	Variable utilizada para recorrer el arreglo en la búsqueda lineal

3. ALGORITMO PASO A PASO

1. Inicio
2. Declarar arreglo productos de dimensión 5
 - 2.1 Inicializar arreglo con nombres de productos
3. Declarar arreglo precios de dimensión 5
 - 3.1 Inicializar arreglo con los respectivos precios
4. Pedir nombre del producto
5. Guardar nombre en una variable
6. Inicializar un ciclo for que inicie en 0 con un recorrido mientras sea menor que 5
7. Comparar variable nombre con producto [i]
 - 7.1 Si existe coincidencia => nombre == producto[i]
 - 7.1.1 Guardar índice
 - 7.1.2. Imprimir Arreglo Productos[i]
 - 7.1.3. Imprimir Arreglo precio[i]
 - 7.2 Si no:



7.2.1 Imprimir no existe el producto

10. Fin

6. Romper ciclo

4. PSEUDOCÓDIGO EN PSEINT

Algoritmo Busqueda_Lineal_Productos

```
// Declaracion de variables
Definir productos Como Cadena
Definir precios Como Real
Definir buscar Como Cadena
Definir i Como Entero
Definir encontrado Como Lógico
// Dimension de arreglos
Dimensionar productos(5)
Dimensionar precios(5)
// Asignacion de datos
productos[0] <- 'Laptop'
productos[1] <- 'Mouse'
productos[2] <- 'Teclado'
productos[3] <- 'Monitor'
productos[4] <- 'Impresora'
precios[0] <- 850.50
precios[1] <- 25.75
precios[2] <- 45.00
precios[3] <- 300.99
precios[4] <- 150.25
// Solicitar producto a buscar
Escribir 'Ingrese el nombre del producto a buscar:'
Leer buscar
encontrado <- Falso
// Busqueda lineal
Para i<-0 Hasta 4 Hacer
    Si productos[i]=buscar Entonces
        Escribir 'Producto encontrado'
        Escribir 'Nombre: ', productos[i]
        Escribir 'Precio: $', precios[i]
        encontrado <- Verdadero
    // Salir del ciclo
    i <- 5
FinSi
FinPara
// Mensaje si no existe
Si encontrado=Falso Entonces
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir 'El producto no existe.'

FinSi

FinAlgoritmo

5. DIAGRAMA DE FLUJO

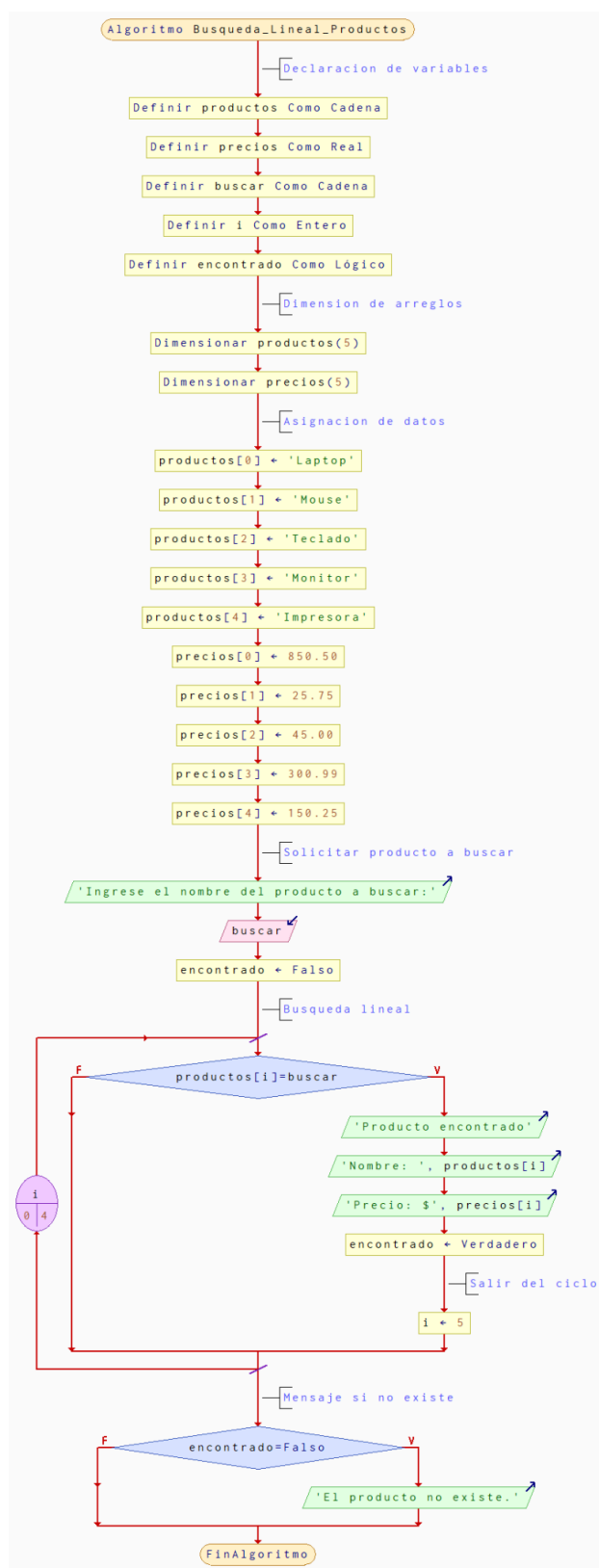


Figura 1. Diagrama de flujo del algoritmo de Búsqueda lineal de productos . Elaboración propia: Atencia J., Barrionuevo J., Rocha C., Segovia J. (2026).

6. CÓDIGO EN JAVA



```
import java.util.Scanner;

public class BusquedaLinealProductos {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] productos = new String[5];
        double[] precios = new double[5];
        int opcion;
        String buscar;
        boolean encontrado;
        // Ingreso de productos y precios
        for (int i = 0; i < 5; i++) {
            System.out.print("Ingrese el nombre del producto " + (i + 1) + ": ");
            productos[i] = sc.nextLine();

            System.out.print("Ingrese el precio del producto: ");
            precios[i] = sc.nextDouble();
            sc.nextLine();
        }
        do {
            System.out.println("\n===== MENU =====");
            System.out.println("1. Buscar producto");
            System.out.println("2. Finalizar programa");
            System.out.print("Seleccione una opcion: ");
            opcion = sc.nextInt();
            sc.nextLine();
            switch (opcion) {
                case 1:
                    encontrado = false;
                    System.out.print("\nIngrese el producto a buscar: ");
                    buscar = sc.nextLine();
                    // Búsqueda lineal
                    for (int i = 0; i < 5; i++) {
                        if (productos[i].equalsIgnoreCase(buscar)) {
                            System.out.println("\nProducto encontrado");
                            System.out.println("Nombre: " + productos[i]);
                            System.out.println("Precio: $" + precios[i]);
                            encontrado = true;
                            break;
                        }
                    }
                    if (!encontrado) {
                        System.out.println("\nEl producto no existe.");
                    }
                    break;
                case 2:
                    System.out.println("\nPrograma finalizado.");
            }
        } while (opcion != 2);
    }
}
```




```
        break;
    default:
        System.out.println("\nOpcion invalida.");
    }
} while (opcion != 2);
sc.close();
}
}
```

7. PRUEBAS DE SOLUCIÓN

```
Ingrese el nombre del producto 1: Protector Solar
Ingrese el precio del producto: 15.75
Ingrese el nombre del producto 2: Crema
Ingrese el precio del producto: 5.25
Ingrese el nombre del producto 3: Coca Cola
Ingrese el precio del producto: 0.50
Ingrese el nombre del producto 4: Vajillas
Ingrese el precio del producto: 5.50
Ingrese el nombre del producto 5: Snack
Ingrese el precio del producto: 1.50

===== MENU =====
1. Buscar producto
2. Finalizar programa
Seleccione una opcion: 1

Ingrese el producto a buscar: Crema

Producto encontrado
Nombre: Crema
Precio: $5.25
```

Figura 2. Pruebas de solución con producto existente. Elaboración propia: Atencia J., Barrionuevo J., Rocha C., Segovia J. (2026).

```
===== MENU =====
1. Buscar producto
2. Finalizar programa
Seleccione una opcion: 1

Ingrese el producto a buscar: Batidor

El producto no existe.

===== MENU =====
1. Buscar producto
2. Finalizar programa
Seleccione una opcion: 2
```

Figura 3. Pruebas de solución con producto inexistente. Elaboración propia: Atencia J., Barrionuevo J., Rocha C., Segovia J. (2026).



Ejercicio 2: Prevención de Buffer Overflow

Realizar un programa en C++ o Java que muestre los elementos de un arreglo sin provocar desbordamiento de memoria.

Indicaciones

Crear un arreglo con 6 números enteros.

Utilizar un ciclo for para recorrer el arreglo.

Validar correctamente los límites del ciclo.

Mostrar cada posición y su valor.

Agregar un comentario explicando qué ocurriría si el ciclo supera el tamaño del arreglo.

1. ANÁLISIS DEL PROBLEMA

El objetivo del programa es mostrar los elementos almacenados dentro de un arreglo de 6 números enteros utilizando un ciclo for, evitando errores relacionados con el desbordamiento de memoria (*buffer overflow*).

Para lograrlo, se debe crear un arreglo con tamaño fijo de 6 posiciones e inicializarlo con valores enteros. Posteriormente, el programa recorrerá cada posición del arreglo utilizando un ciclo for.

Es importante controlar correctamente los límites del ciclo, ya que los arreglos comienzan desde la posición 0. En este caso, las posiciones válidas van desde 0 hasta 5.

Durante el recorrido, el programa mostrará:

- La posición del arreglo.
- El valor almacenado en dicha posición.

Si el ciclo intenta acceder a una posición mayor al tamaño permitido del arreglo, el programa podría:

- Leer datos inválidos.
- Generar resultados incorrectos.
- Producir errores de ejecución o desbordamiento de memoria.

2. DECLARACIÓN DE VARIABLES

Tipo de dato	Nombre de la variable	Descripción
int	numeros[6]	Arreglo de tipo entero que almacena 6 números enteros en memoria
int	i	Variable de control utilizada en el ciclo for para recorrer cada posición del arreglo
int	tamano	Variable que representa la cantidad total de



		elementos del arreglo y ayuda a controlar los límites del recorrido
--	--	---

3. ALGORITMO PASO A PASO

1. Inicio
2. Declarar arreglo de dimensión 6
 - 2.1 Inicializar arreglo con 6 números enteros
3. Inicializar un ciclo for que inicie en 0 con un recorrido mientras sea menor que 6
4. Imprimir numeros[i]

10. Fin

4. PSEUDOCÓDIGO EN PSEINT

Proceso Prevencion_Buffer_Overflow

Definir nombre Como Cadena

Definir cantidad Como Entero

Escribir "Ingrese un nombre de maximo 20 caracteres:"

Leer nombre

cantidad <- longitud(nombre)

Si cantidad <= 20 Entonces

Escribir "Nombre guardado correctamente"

Escribir "Nombre ingresado: ", nombre

Escribir "Cantidad de caracteres: ", cantidad

SiNo

Escribir "ERROR: El nombre supera el limite permitido"

Escribir "Caracteres ingresados: ", cantidad

FinSi

FinProceso

5. DIAGRAMA DE FLUJO

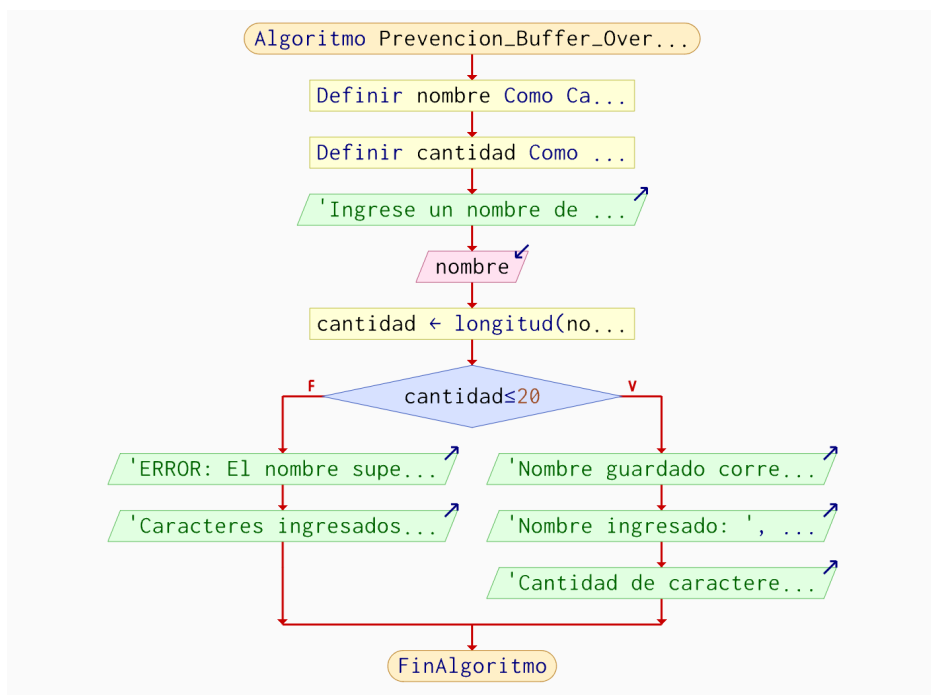


Figura 4. Diagrama de flujo del algoritmo de Prevención de Buffer Overflow . Elaboración propia: Atencia J., Barrionuevo J., Rocha C., Segovia J. (2026).

6. CÓDIGO EN JAVA

```

public class PrevencionOverflow {
    public static void main(String[] args) {

        int[] numeros = {10, 20, 30, 40, 50, 60};

        // Recorrer el arreglo correctamente
        for (int i = 0; i < numeros.length; i++) { //Validacion correcta de los limites del arreglo

            System.out.println("Posicion " + i + ": " + numeros[i]);
        }

        /*
        Si el ciclo supera el tamaño del arreglo,
        el programa intentará acceder a una posición
        inexistente y producirá un error
        llamado ArrayIndexOutOfBoundsException.
        */
    }
}
  
```

7. PRUEBAS DE SOLUCIÓN



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
Posicion 0: 10  
Posicion 1: 20  
Posicion 2: 30  
Posicion 3: 40  
Posicion 4: 50  
Posicion 5: 60
```

Figura 5. Prueba de solución ejercicio 2. Elaboración propia: Atencia J., Barrionuevo J., Rocha C., Segovia J. (2026).